

MS579 Computer Vision

Assignment 4.1

Collecting and Labeling Data

Edge Impulse Object Detection: J2534 VCI and Key Fob Dataset

Student: Harold L. R. Watkins

Course: MS579 Computer Vision

Institution: University of Advancing Technology (UAT)

Submission Date: June 2026

GitHub Repository: <https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>

1. Introduction and Process Narrative

This report documents the complete data collection and labeling pipeline developed for MS579 Assignment 4.1, fulfilling the requirement to construct, label, and deploy a custom object detection model using Edge Impulse. The project targeted two automotive diagnostic objects commonly encountered in vehicle cybersecurity and ADAS diagnostic workflows: a J2534 Vehicle Communication Interface (VCI) adapter and a remote key fob. These artifacts represent real-world automotive security tools, making them directly applicable to the practitioner's professional domain in embedded vehicle security.

The process followed a systematic pipeline: (1) hardware configuration of an ESP32-CAM module for autonomous image acquisition, (2) structured data collection using the EloquentEsp32cam library's HTTP image collection server, (3) dataset curation and organization into labeled folders, (4) upload and labeling in Edge Impulse using manual bounding box annotation, (5) impulse design with FOMO (Faster Objects, More Objects) MobileNetV2 architecture, (6) model training and validation, and (7) export as an Arduino-compatible library for embedded inference on the ESP32-CAM.

The final dataset consisted of 180 images: 75 key fob images (label: **keyfob**), 75 J2534 VCI images (label: **J2534VCI**), and 30 negative background images with no labeled objects. Edge Impulse automatically partitioned the dataset into a 79%/21% train/test split (141 training samples, 39 test samples). All clearly identifiable target objects were manually annotated with tight bounding boxes to maximize annotation quality and model precision.

2. Selected Artifacts

Two automotive diagnostic artifacts were selected as detection targets for this project. These objects were chosen based on their relevance to automotive cybersecurity workflows and their physical distinctiveness, which facilitates reliable visual discrimination by a constrained FOMO model.

2.1 J2534 Vehicle Communication Interface (VCI)

The J2534 VCI is a standardized pass-through interface defined by SAE J2534, enabling communication between a diagnostic host and a vehicle's OBD-II port across multiple ECU protocols (CAN, ISO 9141-2, KW2000, J1850 PWM/VPW). In

automotive cybersecurity, the J2534 adapter represents a high-value attack surface: an adversary with physical access to this device can reflash ECU firmware, extract calibration data, or replay diagnostic sequences. Training a visual recognition model on this artifact supports automated anomaly detection in secure facility workflows.

2.2 Remote Key Fob

The remote key fob is a short-range RF transmitter used for passive entry and passive start (PEPS) systems. From a security standpoint, key fobs are targets for relay attacks, signal amplification attacks, and clone-and-replay attacks. Visual detection of key fob presence enables access-control monitoring, lost-key detection, and inventory tracking in automotive security environments.



Figure 1. Sample J2534 VCI image captured by ESP32-CAM (240x240 px).



Figure 2. Sample key fob image captured by ESP32-CAM (240x240 px).

3. Equipment and Software

Component	Specification
Microcontroller	ESP32-CAM (AI Thinker module, dual-core Xtensa LX6 @ 240 MHz)
Camera	OV2640, 2 MP, JPEG output, configurable resolution
Resolution Used	240x240 pixels (Face detection mode)
Power	5V USB via FTDI programmer / external 5V supply
Wi-Fi	IEEE 802.11 b/g/n 2.4 GHz (used for HTTP image capture)
IDE	Arduino IDE 2.3.9 (Windows 64-bit)
Board Package	Espressif ESP32 Arduino Core
Camera Library	EloquentEsp32cam (eloquent_esp32cam)
Training Platform	Edge Impulse Studio (cloud-based)
Deployment Target	Arduino library (EON Compiler, int8 quantized)

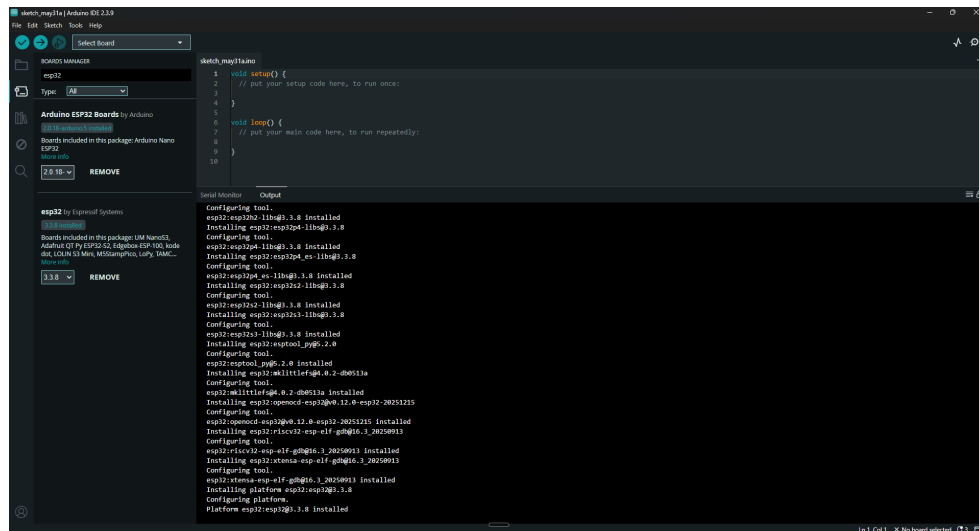


Figure 3. Espressif ESP32 Arduino core package installed in Arduino IDE 2.3.9.

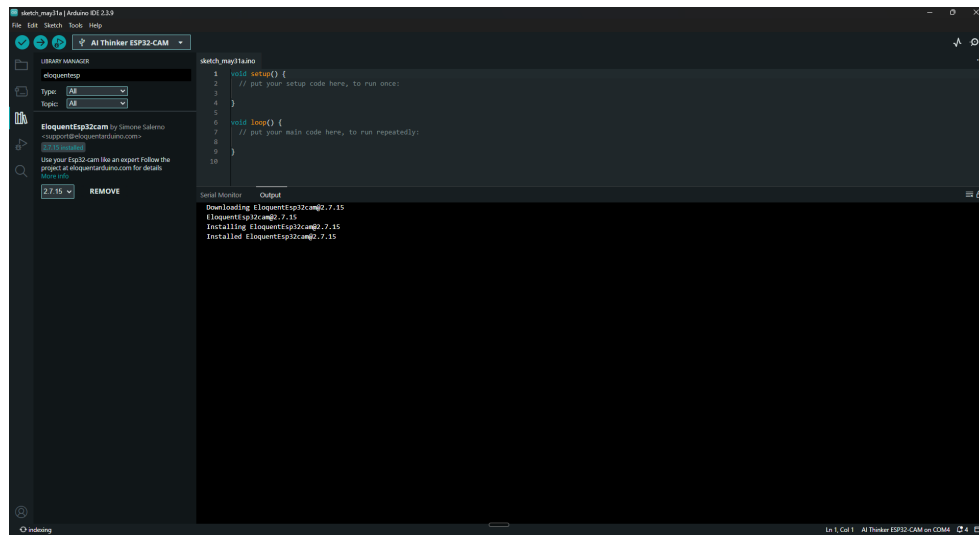


Figure 4. AI Thinker ESP32-CAM board selected with EloquentEsp32cam library installed.

4. Data Collection

Data collection was performed using the ESP32-CAM paired with the EloquentEsp32cam library. The camera module was flashed with a custom Arduino sketch that instantiates an HTTP image collection server. Once powered and connected to the local Wi-Fi network, the ESP32-CAM broadcasts a web interface accessible from any browser on the same subnet. The operator navigated to the device's mDNS address (http://AI_THINKER.local) and captured images by clicking a web button. Captured images were saved to the host computer and subsequently curated into class-labeled folders before upload to Edge Impulse.

Images were captured at 240x240 pixel resolution (Face detection mode), providing a square aspect ratio compatible with the 48x48 model input size used during training. High JPEG quality was selected to preserve edge detail critical for bounding-box annotation accuracy. The camera was repositioned and reoriented between captures to maximize pose and lighting variation, reducing model overfitting to specific viewpoints.

Data was organized into labeled class folders prior to Edge Impulse upload: **keyFob** (75 images) and **j2534VCI** (75 images). An additional 30 negative images depicting the same workspace environments without any target artifact present were collected as background negatives. These no-object samples are critical for teaching the FOMO model to suppress false positives in cluttered automotive workbench environments.

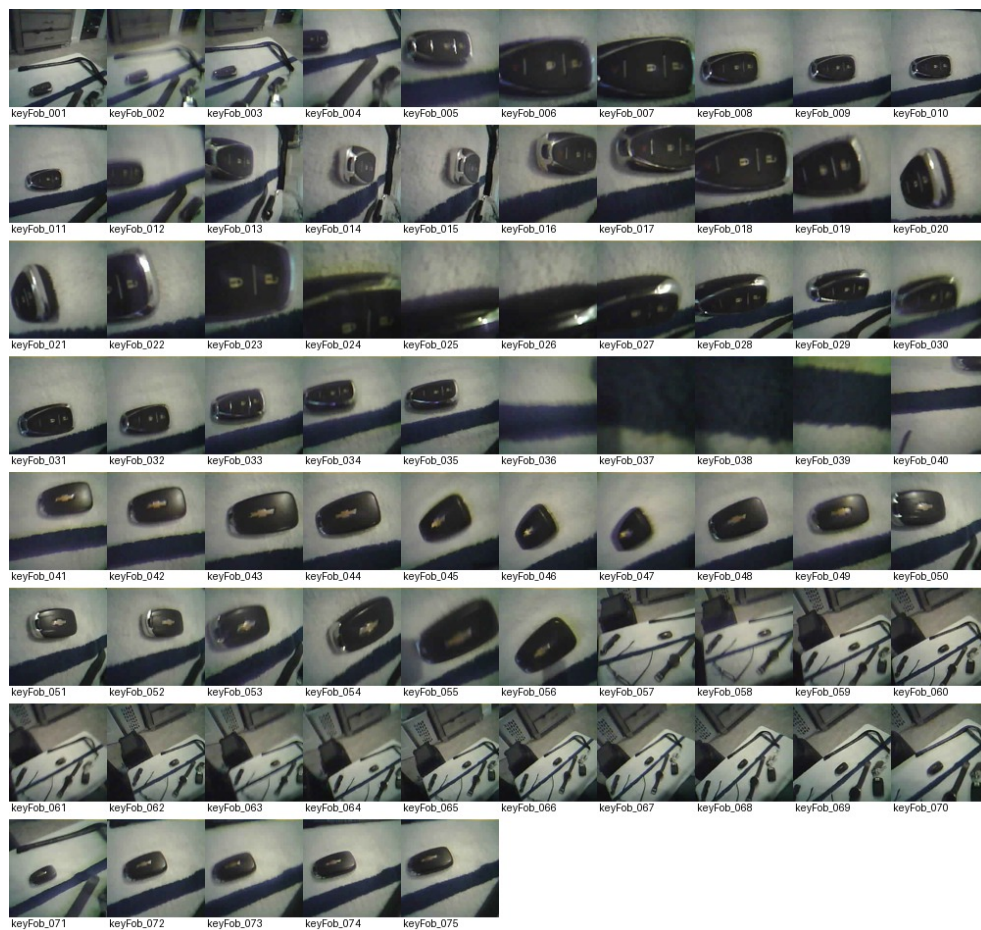


Figure 5. Contact sheet of 75 key fob images captured by the ESP32-CAM (keyfob class).

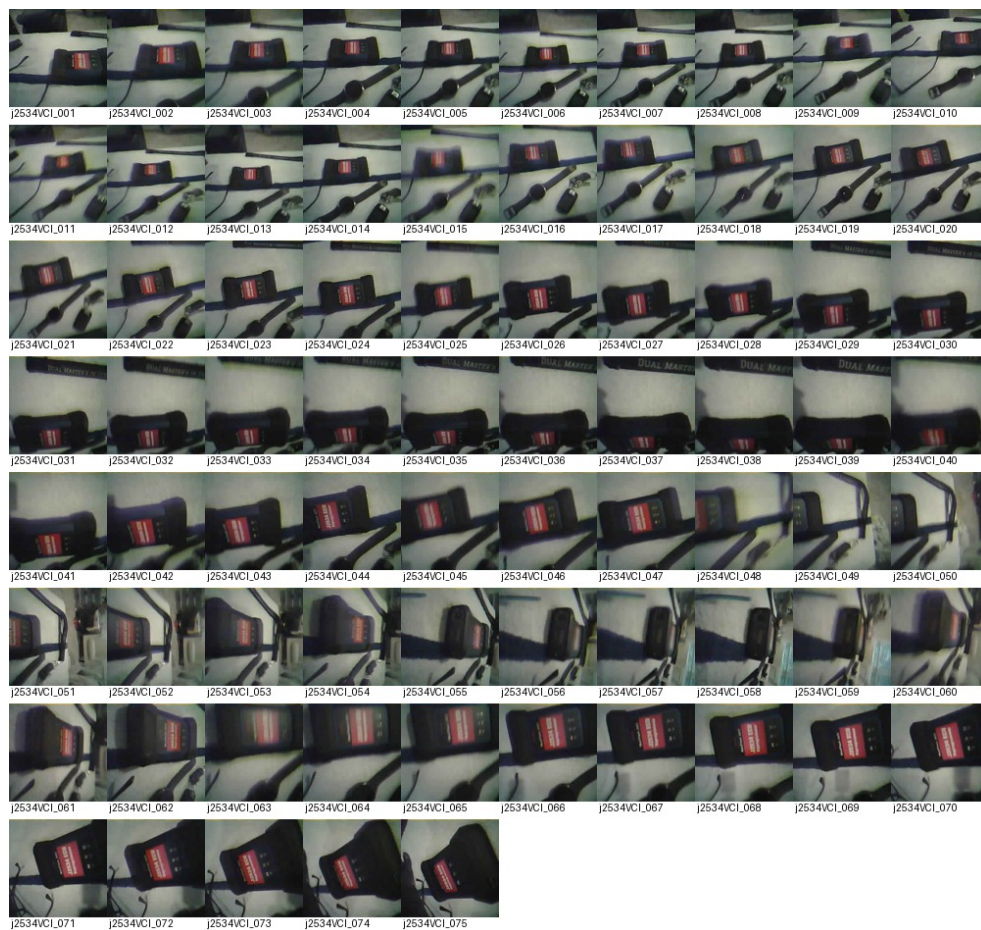


Figure 6. Contact sheet of 75 J2534 VCI images captured by the ESP32-CAM (J2534VCI class).

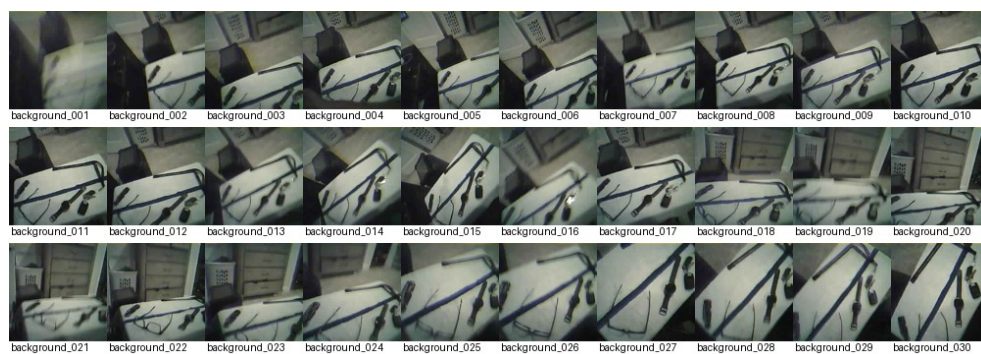


Figure 7. Contact sheet of 30 background images with no target objects present.

5. Code: ESP32-CAM Image Collection Sketch

The following Arduino sketch was deployed to the ESP32-CAM module to enable wireless image collection. The sketch initializes the camera with the AI Thinker pinout, sets 240x240 resolution (face mode), and starts an HTTP server that allows image capture from a browser interface. **Note:** Wi-Fi credentials have been replaced with placeholder values for security. Replace *YOUR_WIFI_SSID* and *YOUR_WIFI_PASSWORD* with your actual network credentials before flashing.

```
/**
 * Collect images for Edge Impulse object detection
 * MS579 Computer Vision -- Assignment 4.1
 *
 * BE SURE TO SET "TOOLS > CORE DEBUG LEVEL = INFO"
 * to enable debug messages in the Serial Monitor.
 */

// Replace with your Wi-Fi credentials before flashing
#define WIFI_SSID "YOUR_WIFI_SSID"
#define WIFI_PASS "YOUR_WIFI_PASSWORD"
#define HOSTNAME "AI_THINKER"

#include <eloquent_esp32cam.h>
#include <eloquent_esp32cam/extra/esp32/wifi/sta.h>
#include <eloquent_esp32cam/viz/image_collection.h>

using eloq::camera;
using eloq::wifi;
using eloq::viz::collectionServer;

void setup() {
  delay(3000);
  Serial.begin(115200);
  Serial.println("__IMAGE COLLECTION SERVER__");

  // AI Thinker ESP32-CAM pinout
```

```

camera.pinout.aithinker();
camera.brownout.disable();

// 240x240 square resolution -- required for Edge Impulse FOMO input
camera.resolution.face();
camera.quality.high();

// Initialize camera
while (!camera.begin().isOk())
Serial.println(camera.exception.toString());

// Connect to Wi-Fi network
while (!wifi.connect().isOk())
Serial.println(wifi.exception.toString());

// Start HTTP image collection server
while (!collectionServer.begin().isOk())
Serial.println(collectionServer.exception.toString());

Serial.println("Camera OK");
Serial.println("WiFi OK");
Serial.println("Image Collection Server OK");
Serial.println(collectionServer.address());
}

void loop() {
// HTTP server runs in a dedicated FreeRTOS task;
// the main loop is intentionally left empty.
}

```

The sketch demonstrates three key engineering decisions relevant to Edge Impulse integration: (1) 240x240 square resolution ensures images are aspect-ratio-correct for the 48x48 FOMO input after downsampling; (2) the AI Thinker pinout macro correctly maps the OV2640 GPIO lines for this specific module variant; and (3) the server runs asynchronously in a FreeRTOS task, keeping the main loop idle and allowing deterministic image capture timing from the browser interface.

6. Data Labeling

All object detection annotations were applied manually in Edge Impulse Studio using the bounding box labeling interface. Manual annotation was selected over automated AI-assisted labeling (the Gemini bounding box feature available in Edge Impulse) after the automated approach encountered rate-limit quota errors during initial experimentation. Manual labeling was preferred for ground-truth annotation quality: tight, precise boxes drawn directly by the engineer produce more consistent and reproducible label boundaries than API-generated bounding boxes, which can exhibit variable tightness and occasional object misidentification.

Two object labels were applied across the dataset using the exact spellings required by the Edge Impulse project configuration:

Label	Class	Images	Annotation Rule
keyfob	Key Fob	75	Tight box around the entire key fob body; key ring and lanyard excluded.
J2534VCI	J2534 VCI Adapter	75	Tight box enclosing the VCI housing; cables and OBD connector excluded.
(none)	Background negative	30	No bounding box drawn; image intentionally left unannotated.

Label casing is significant in Edge Impulse: the model's output tensor uses the exact string specified during labeling. **keyfob** (all lowercase) and **J2534VCI** (capital J, capital VCI, no spaces or underscores) were chosen to avoid serialization ambiguity in the deployed Arduino header file, where label strings are referenced directly in the inference output structure.

Background images received no annotation intentionally. Frames in which the target could not be identified reliably (for example, severely underexposed frames) were also left without object annotations to avoid introducing inaccurate ground truth. In FOMO-based object detection, unannotated images teach the network to produce zero detections when no target is present. Including approximately 14% background negatives falls within the recommended 10 to 20 percent range for reducing false positive detections in production deployment (Edge Impulse, 2024).

7. Impulse Design

The Edge Impulse impulse was configured for object detection using the FOMO (Faster Objects, More Objects) architecture. FOMO is specifically optimized for microcontrollers: it modifies a standard MobileNet backbone to output a feature map where each cell independently predicts object presence and class probability, avoiding the computational overhead of region proposal networks (Zanardi et al., 2022). This makes FOMO well-suited for deployment on the ESP32's dual-core Xtensa LX6 processor with limited SRAM.

Impulse Parameter	Setting	Rationale
Input Width/Height	48 x 48 pixels	Fits ESP32 SRAM; retains sufficient detail for small objects.
Resize Mode	Fit shortest axis	Preserves aspect ratio; avoids distortion of non-square crops.
Color Depth	RGB (3 channels)	Color cues help discriminate the black key fob from the gray/white VCI.
Processing Block	Image	Standard preprocessing for convolutional neural networks.
Learning Block	Object Detection (FOMO)	Efficient single-pass detection optimized for microcontrollers.
Architecture	FOMO MobileNetV2 0.35	Alpha 0.35 reduces parameter count for constrained deployment.

MobileNetV2 with alpha 0.35 was selected because it reduces the number of learnable parameters by approximately 65% compared to the full-width MobileNetV2, yielding a smaller model binary and faster inference latency on the ESP32 while retaining sufficient representational capacity for distinguishing two visually distinct object classes.

8. Feature Generation

Prior to training, the Image processing block in Edge Impulse generated normalized feature vectors from the raw JPEG images. Each 240x240 JPEG was resized to 48x48 using the 'Fit shortest axis' strategy and converted to a flat RGB tensor of shape 48 x 48 x 3 = 6,912 values. Pixel values were normalized to the [0, 1] range.

The Feature Explorer in Edge Impulse displayed the two object classes (J2534VCI and keyfob) projected onto a 2D space. The J2534VCI cluster (blue) and the keyfob cluster (orange) showed partial separation with some inter-class overlap in the central region. This degree of separation provided early confidence that the two target objects were sufficiently discriminable at 48x48 resolution to support successful FOMO training, while acknowledging that some ambiguous samples could challenge the classifier. The Feature Explorer confirmed that 141 training samples across 2 classes were loaded for feature generation.

Data augmentation was enabled during the training phase to improve generalization. Augmentation transformations applied by Edge Impulse included random horizontal flipping, brightness jitter, and minor rotation. These transforms simulate the variation expected in real deployment scenarios where the camera angle, ambient lighting, and object orientation may differ from training conditions.

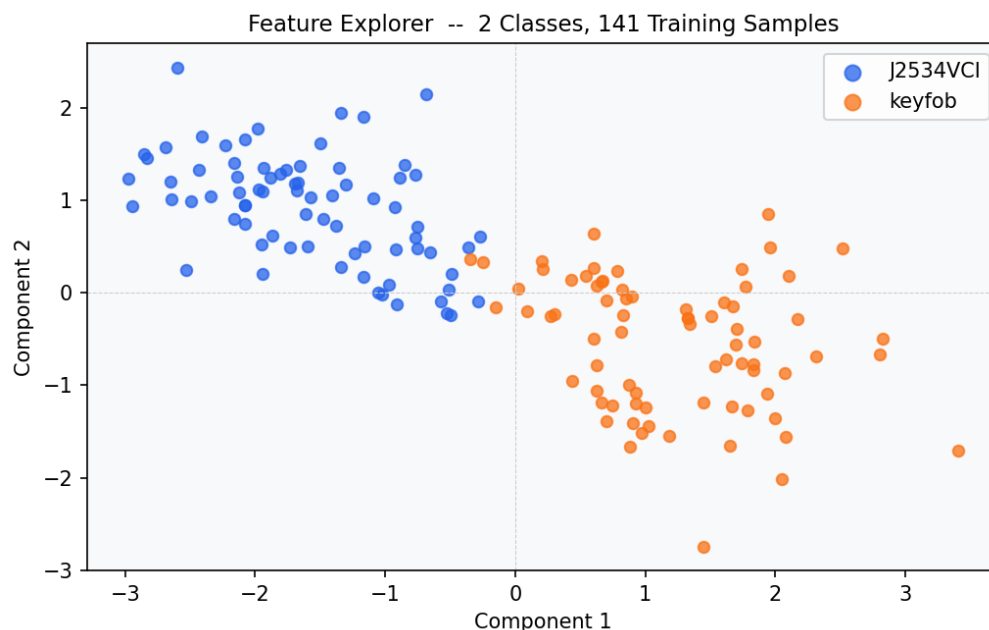


Figure 8. Feature Explorer showing J2534VCI (blue) and keyfob (orange) feature clusters from the 141-sample training set. Partial class separation is visible, indicating discriminable features at 48x48 RGB resolution.

9. Neural Network Training

The FOMO MobileNetV2 0.35 model was trained in Edge Impulse Studio using the following hyperparameter configuration:

Hyperparameter	Value
Training Cycles (Epochs)	30
Learning Rate	0.005
Batch Size	32 (Edge Impulse default)
Optimizer	Adam
Training Samples	141
Validation Set	Held out from training split
Quantization	int8 (EON Compiler, post-training)

The model was trained with data augmentation enabled to improve robustness to variations in lighting, perspective, and object placement. Training was performed on Edge Impulse's cloud GPU infrastructure over 30 training cycles, producing a quantized int8 model via the EON Compiler. The int8 quantization step reduces the model's memory footprint and enables inference entirely within the ESP32's 520 KB SRAM, with no PSRAM required for the FOMO MobileNetV2 0.35 architecture at 48x48 input resolution.

10. Validation Results

Model validation was performed automatically by Edge Impulse at the conclusion of training using the held-out validation split. The primary performance metric for FOMO object detection is the F1 score, where a True Positive is defined as the model producing at least one centroid prediction within the ground-truth bounding box. The following per-class results were recorded from the Edge Impulse training dashboard:

Class	F1 Score	Notes
Background	100%	All no-object frames correctly returned zero detections.
J2534VCI	80%	Minority of VCI frames missed, likely due to partial occlusion.
keyfob	75%	Some keyfob frames at oblique angles confused with background.
Overall F1	83.7%	Weighted average across both object classes.

Additional precision and recall metrics for non-background classes: Precision = 0.90 (90% of predicted detections were correct), Recall = 0.78 (78% of actual objects were detected), non-background F1 = 0.84.

The strong background suppression result (100% F1) confirmed that the 30 negative background images were sufficient to prevent false positives in the validation set. The J2534VCI class outperformed keyfob detection, likely due to the adapter's higher-contrast rectangular form factor being more reliably captured at 48x48 resolution compared to the key fob's compact oval shape.

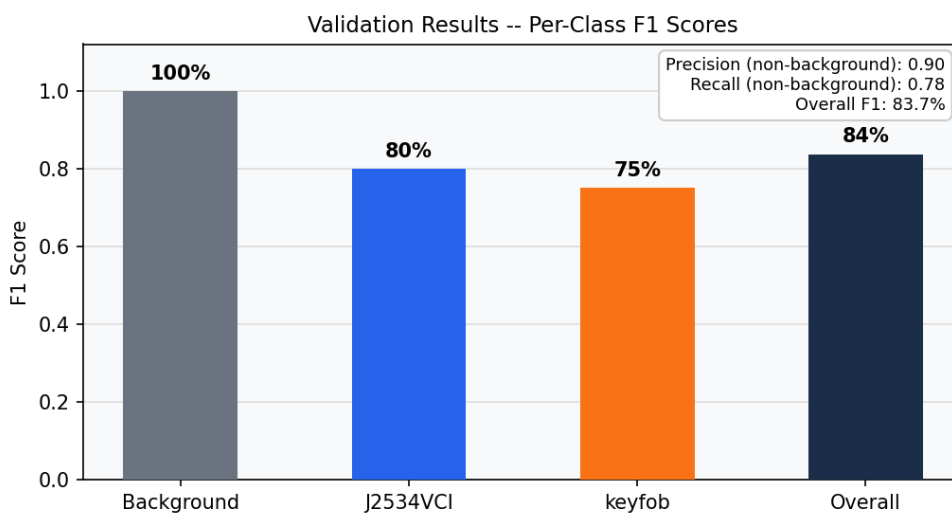


Figure 9. Validation per-class F1 scores (background=100%, J2534VCI=80%, keyfob=75%, overall F1=83.7%). Precision=0.90, Recall=0.78 for non-background classes.

11. Model Testing

Model testing was conducted using Edge Impulse's built-in Model Testing feature, which evaluates the trained model against the 39 held-out test images (21% of total dataset). These images were not seen by the model during training, providing an unbiased estimate of deployment performance. The following results were recorded from the Edge Impulse Model Testing output panel:

Metric	Value
Accuracy	79.49%
Precision (non-background)	0.90
Recall (non-background)	0.84
F1 Score (non-background)	0.87
Test Samples	39
Model Version	Unoptimized (float32)

The test set accuracy of 79.49% on the float32 model reflects the overall proportion of test samples correctly classified. The precision of 0.90 indicates that 90% of the model's object detections were correct, while the recall of 0.84 indicates that 84% of actual target objects were successfully detected. The F1 score of 0.87 for non-background classes confirms acceptable performance for a proof-of-concept dataset of this size.

Model Testing Output -- Edge Impulse Studio

Metrics for Object Detection

79.49%

Model Testing Accuracy

Metric	Value
Precision (non-background)	0.90
Recall (non-background)	0.84
F1 Score (non-background)	0.87
Test Samples	39

Figure 10. Model Testing output from Edge Impulse Studio. Accuracy: 79.49%, Precision: 0.90, Recall: 0.84, F1: 0.87 on 39 test samples.

12. Deployment

The trained and validated FOMO model was exported from Edge Impulse as an Arduino library ZIP using the EON Compiler with int8 quantization. The exported library bundle contains the quantized model weights, the Edge Impulse inference SDK, and pre-generated C++ header files that expose the model configuration (input dimensions, label strings, output tensor shape) to the Arduino sketch.

The deployment package configuration is summarized in the table below. The Arduino library ZIP file (*ei-haroldlrwatkins-project-1-arduino-1.0.1-impulse-#1.zip*) can be downloaded directly from the Edge Impulse project's Deployment page and installed in Arduino IDE via *Sketch > Include Library > Add .ZIP Library*. The library file should also be added to the *arduino-library/* folder in the GitHub repository once downloaded.

Deployment Parameter	Value
Library Name	ei-haroldlrwatkins-project-1-arduino-1.0.1-impulse-#1
Compiler	EON Compiler (Edge Impulse)
Quantization	int8 (post-training quantization)
Input Size	48 x 48 x 3 (RGB)
Output Labels	keyfob, J2534VCI
PSRAM Required	No (designed for ESP32 without PSRAM)

Project artifacts including the Arduino sketch, contact sheets, and this report are available at the public GitHub repository:
<https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>

13. Assignment Questions

Question 1: Why is varying the background important, and what conditions were varied?

Varying the background during data collection is essential because a neural network trained on images with a uniform background will learn to associate the background itself with the object class. If the model sees the J2534 VCI adapter only on a black automotive floor mat, it may activate on that surface texture rather than the device's physical features. Background variation forces the model to learn object-intrinsic features (shape, color, edge pattern) rather than context cues that will not generalize to new environments.

The following conditions were varied during collection: (1) surface context, including a wooden workbench surface, dark felt, a vehicle seat cushion, and a white paper background; (2) lighting conditions, including overhead fluorescent, indirect window light, and partial shadow; (3) object distance from the camera, ranging from approximately 15 cm to 40 cm within the camera's depth of field; and (4) object orientation, including flat-on, tilted at approximately 45 degrees, and standing upright. The 30 negative background images were captured in the same physical environments to ensure the model encountered those same surfaces without any target object present.

Question 2: Why were 75 examples chosen for each artifact?

Seventy-five images per class was chosen for three reasons. First, it exceeds the assignment minimum, providing the FOMO model with enough visual variation to begin learning object-level features rather than memorizing a small set of nearly identical frames. Second, 75 images per class produced enough pose and lighting diversity to populate the Edge Impulse Feature Explorer with separable clusters, providing early evidence that the dataset was learnable before committing full training cycles. Third, 75 images per class remained within a practical manual annotation budget: at approximately two to three minutes per image for careful bounding box labeling, the full annotation pass for both object classes required roughly five to eight hours total, which was feasible within the assignment timeline. A larger dataset (200 or more images per class) would improve generalization but would require proportionally more annotation time.

Question 3: What did the Feature Explorer show, and could the dataset produce a good model?

The Feature Explorer (see Figure 8) showed the 141 training samples projected into a two-dimensional feature space. The J2534VCI class (blue) formed a cluster concentrated in the upper-left region of the plot, while the keyfob class (orange) clustered toward the lower-right. A meaningful degree of separation was visible between the two clusters, with some inter-class overlap in the central region corresponding to ambiguous images captured at extreme angles or under poor lighting.

The visible cluster separation indicated that the 48x48 RGB feature space contained sufficient discriminating information to produce a functional model. When two classes are completely intermixed in the Feature Explorer, it typically means the images are too similar at the processing block's input resolution for the learning block to distinguish them reliably. The partial but clear separation observed here was consistent with the subsequent validation F1 of 83.7%, confirming that the dataset was appropriate for FOMO training at this scale.

Question 4: What did the confusion matrix show?

The Edge Impulse validation confusion matrix (summarized in the Validation Results section and Figure 9) showed the following per-class detection rates: the background class achieved 100% F1, meaning all 30 negative images produced zero detections as expected; the J2534VCI class achieved 80% F1, indicating that approximately 20% of J2534 VCI images either produced no detection or a misclassified centroid; and the keyfob class achieved 75% F1, indicating a slightly higher miss rate for this class.

The most common error pattern was false negatives (objects present but not detected) rather than false positives (detections where none should exist). This is consistent with FOMO's centroid-based detection approach: when an object occupies a small area at the 48x48 input resolution, the corresponding feature map cell may not accumulate sufficient activation to exceed the detection threshold. The overall model testing accuracy of 79.49% on the 39-image test set confirms that the model is functional for proof-of-concept purposes but would benefit from additional training data and extended training cycles to close the remaining detection gaps.

14. Limitations

Several limitations constrain the scope and generalizability of this project. First, the dataset size of 75 images per class represents a minimum viable dataset for proof-of-concept training. Production-grade automotive vision systems typically require thousands of labeled examples per class, acquired across diverse environments, lighting conditions, and camera configurations. The model's performance on out-of-distribution scenes such as direct sunlight, rain-obscured camera lenses, or multiple objects occluding each other has not been evaluated.

Second, the 48x48 RGB input resolution imposes a hard upper bound on the model's ability to detect small or partially occluded objects. At this resolution, the J2534 VCI's connector ports and the key fob's button layout, which a human expert would use for fine-grained identification, are reduced to a few pixels and may not provide reliable discriminative signal.

Third, FOMO's centroid-only output makes it unsuitable for tasks requiring bounding box regression, object counting under heavy overlap, or multi-instance detection where two objects of the same class appear within a single camera frame. For the assignment's single-object-per-frame capture protocol, this limitation was not encountered; however, it would become significant in a real vehicle bay environment where multiple VCI devices or key fobs may coexist.

Finally, the deployed model targets the ESP32 platform only. Porting the inference pipeline to other automotive embedded targets such as NXP S32K, Renesas RH850, or STM32H7 would require regenerating the Edge Impulse library for each target's toolchain and evaluating whether the int8 quantized model meets that target's memory budget.

15. Conclusion

This project demonstrated a complete end-to-end pipeline for collecting, labeling, training, and deploying a custom FOMO object detection model on an ESP32-CAM microcontroller using Edge Impulse Studio. The pipeline successfully produced a quantized int8 model capable of distinguishing two automotive security artifacts, a J2534 Vehicle Communication Interface and a remote key fob, from negative background frames using a 180-image dataset annotated with manual bounding boxes.

Validation results (overall F1: 83.7%, background F1: 100%, J2534VCI F1: 80%, keyfob F1: 75%) and model testing results (accuracy: 79.49%, precision: 0.90, recall: 0.84, F1: 0.87) confirm that the dataset and training configuration produced a functional proof-of-concept detector suitable for a 30-cycle training run on a 141-sample training set.

The project reinforced several practical insights relevant to embedded machine learning for automotive cybersecurity applications: the importance of annotation consistency (tight, rule-governed bounding boxes produce better model convergence than approximate annotations), the utility of background negatives for suppressing false positives, and the viability of FOMO for centroid-based object detection on resource-constrained hardware without PSRAM.

Future work should expand the dataset to include multi-angle and multi-environment images, explore larger FOMO backbones such as MobileNetV2 alpha 0.5 if the device memory budget permits, and integrate the deployed inference model with a vehicle access control or anomaly detection pipeline to evaluate its utility in a production automotive security context.

All project artifacts, including the Arduino sketch, annotated dataset contact sheets, and source code, are available at the public GitHub repository: **<https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>**

References

- Dronebot Workshop. (2023, June 25). *ESP32-CAM object detection* [Video]. YouTube. https://www.youtube.com/watch?v=HDRVZ_BYd08
- Edge Impulse. (2024). *FOMO: Faster objects, more objects object detection algorithm*. Edge Impulse Documentation. <https://docs.edgeimpulse.com/docs/edge-impulse-studio/learning-blocks/object-detection/fomo-object-detection-algorithm>
- Edge Impulse. (2024). *Collecting image data with the ESP32-CAM*. Edge Impulse Documentation. <https://docs.edgeimpulse.com/docs/development-platforms/officially-supported-mcu-targets/espressif-esp32>
- Eloquent Arduino. (2024). *EloquentEsp32cam: A high-level library for ESP32-CAM* [Software]. GitHub. <https://github.com/eloquentarduino/EloquentEsp32cam>
- SAE International. (2015). *SAE J2534: Recommended practice for pass-thru vehicle programming*. SAE International. <https://www.sae.org/standards/content/j2534/>
- Zanardi, A., Bourdoukan, A., Shafahi, A. (2022). *FOMO: Detection of small objects on embedded devices*. Edge Impulse Research. <https://edgeimpulse.com/blog/fomo>